

PROJET MACHINE LEARNING

PRÉDICTION DU DÉFAUT DE REMBOURSEMENT D'UN PRÊT IMMOBILIER (MORTGAGE)

I. COMPRÉHENSION DE LA PROBLÉMATIQUE

1) CONTEXTE ET ENJEU MÉTIER

Ce présent rapport s'inscrit dans le cadre du Machine Learning appliqué au secteur bancaire et financier. En effet, le crédit immobilier est l'un des produits financiers les plus exposés au risque de défaut de paiement, notamment en raison des volumes de capitaux engagés et de la longue durée des contrats. Pour une institution bancaire, la défaillance d'un emprunteur engendre d'une part, une perte financière directe liée au non remboursement et aux intérêts manqués, et d'autre part, aux coûts opérationnels supplémentaires. Face à ce double impact, tout l'enjeu stratégique des banques réside donc dans la réduction du risque de défaut de paiement et dans la bonne gestion de l'octroi du prêt aux clients.

L'objectif est donc de développer un modèle prédictif global capable d'évaluer automatiquement le risque de défaut de remboursement d'un client ayant souscrit à un prêt immobilier, à partir des caractéristiques de son dossier.

Le dataset utilisé est le loan default dataset, un jeu de données couvrant les informations de l'emprunteur (le client), les caractéristiques du prêt et l'historique de crédit.

2) TRADUCTION EN TÂCHE MACHINE LEARNING

Il s'agit d'une classification binaire, s'inscrivant autour de cette problématique : **À partir des caractéristiques d'un dossier de prêt immobilier, est-il possible de prédire si le client sera en défaut de paiement ?**

À partir des caractéristiques d'un dossier (profil de l'emprunteur, caractéristiques du prêt, historique de crédit), le modèle prédit :

- **Status = 1** (défaut de paiement) ou ;
- **Status = 0** (remboursement normal).

Cette distinction est importante pour les établissements bancaires. En effet, Un faux négatif (client en défaut non détecté) représente une perte financière directe pour la banque. Un faux positif (client sain prédit comme défaillant) entraîne un refus injustifié et une perte commerciale. Le modèle doit donc trouver le meilleur équilibre entre ces deux types d'erreurs.

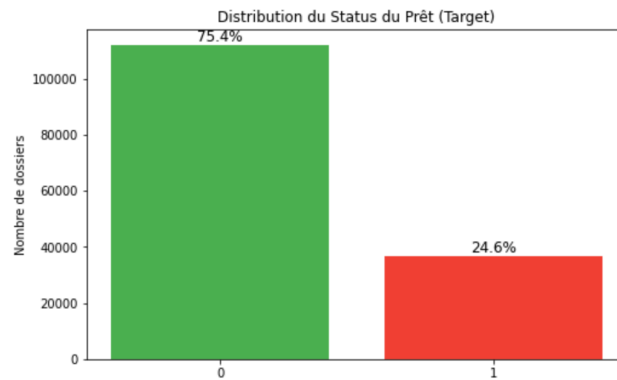
3) ETAT DE L'ART SYNTHÉTIQUE

Pour la prédiction de ce défaut de remboursement on utilise principalement 4 familles d'algorithmes :

- 1- Régression logistique : modèle linéaire de référence ;
- 2- Arbres de décision : robuste aux outliers et détecte les non linéarités ;
- 3- Gradient boosting : offre un bon équilibre entre biais et variance ;
- 4- Réseaux de neurones : puissants sur de grands volumes de données.

Compte tenu de déséquilibre modéré des classes (75,4 % de classe 0 contre 24,6 % de classe 1 : l'accuracy serait donc trompeuse), on utilise 2 métriques principales :

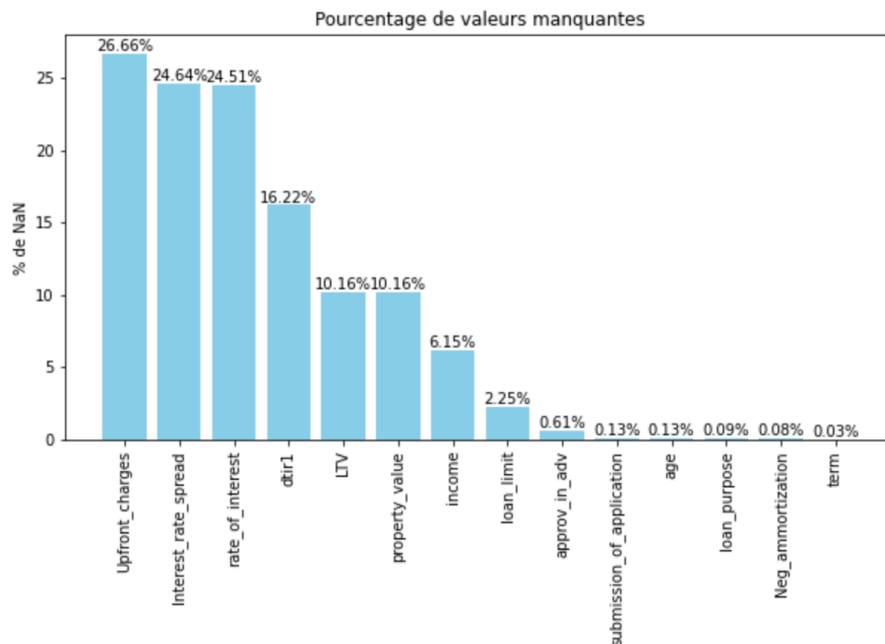
- L'AUC/ROC (Area under the roc curve) : mesure la capacité globale du modèle à discriminer 2 classes. Plus la valeur est proche de 1 plus le modèle est performant ;
- Le F1-score : indicateur d'équilibre entre la précision et le rappel, particulièrement adapté aux classes déséquilibrées.



II. ANALYSE EXPLORATOIRE DES DONNÉES

1) VALEURS MANQUANTES

L'analyse des valeurs manquantes révèle que plusieurs colonnes présentent des valeurs manquantes. Les variables les plus impactées sont : **upfront_charges** et **interest_rate_spread**, **rate_of_interest**, **property_value** et **LTV**. Ces variables auront un fort impact prédictif, la stratégie d'imputation retenue est l'imputation par la médiane, plus robuste que la moyenne face aux distributions asymétriques.

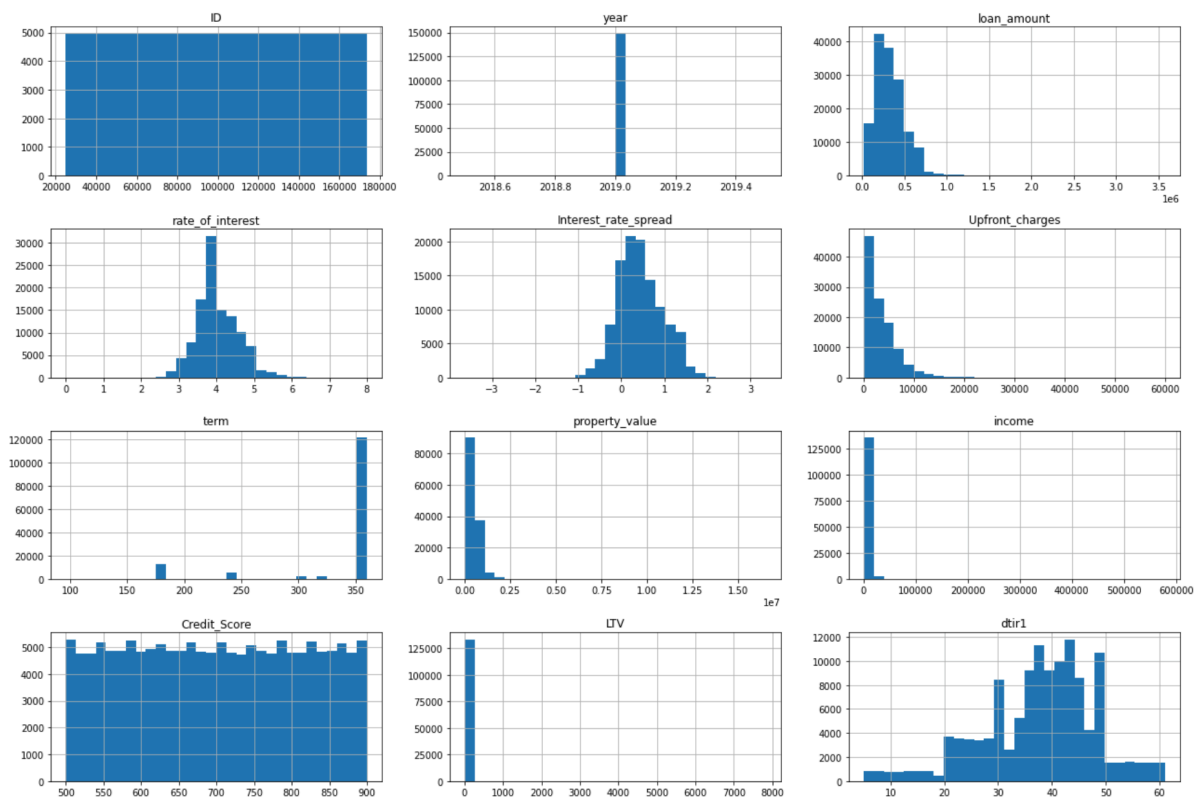


2) DISTRIBUTION DES VARIABLES NUMÉRIQUES

L'analyse de la variable cible Status montre un déséquilibre modéré : (75,4 % de classe 0 contre 24,6 % de classe 1). Ce déséquilibre justifie l'utilisation du F1-score comme métrique principale, et l'utilisation d'une technique de rééchantillonnage (SMOTE, bien que cette option ait finalement été remplacée).

L'analyse des histogrammes des variables numériques révèle plusieurs caractéristiques importantes :

- Asymétrie droite marquée : Les variables financières (loan_amount, property_value, income) sont très étirées vers la droite. Cette asymétrie justifiera le choix du scaler lors du pré-processing ;
- Certaines variables numériques présentent des distributions discrètes ;
- D'autres distributions sont symétriques et quasi normales.



Ces observations justifient l'utilisation d'un standardscaler ou robust scaler pour neutraliser l'effet des valeurs extrêmes et mettre toutes les variables à la même échelle, indispensable pour les algorithmes sensibles aux distances.

3) ANALYSE DES VARIABLES CATÉGORIELLES

Pour mesurer statistiquement la relation entre chaque variable catégorielle et la variable cible, un test du khi-deux a été appliqué à l'ensemble des variables catégorielles en utilisant `pd.crosstab()`.

Les résultats du test du Khi-deux montrent que toutes les variables catégorielles présentent une relation statistiquement significative avec la variable cible. Cette situation s'explique notamment par la grande taille du dataset, qui rend le test très sensible aux faibles différences entre catégories.

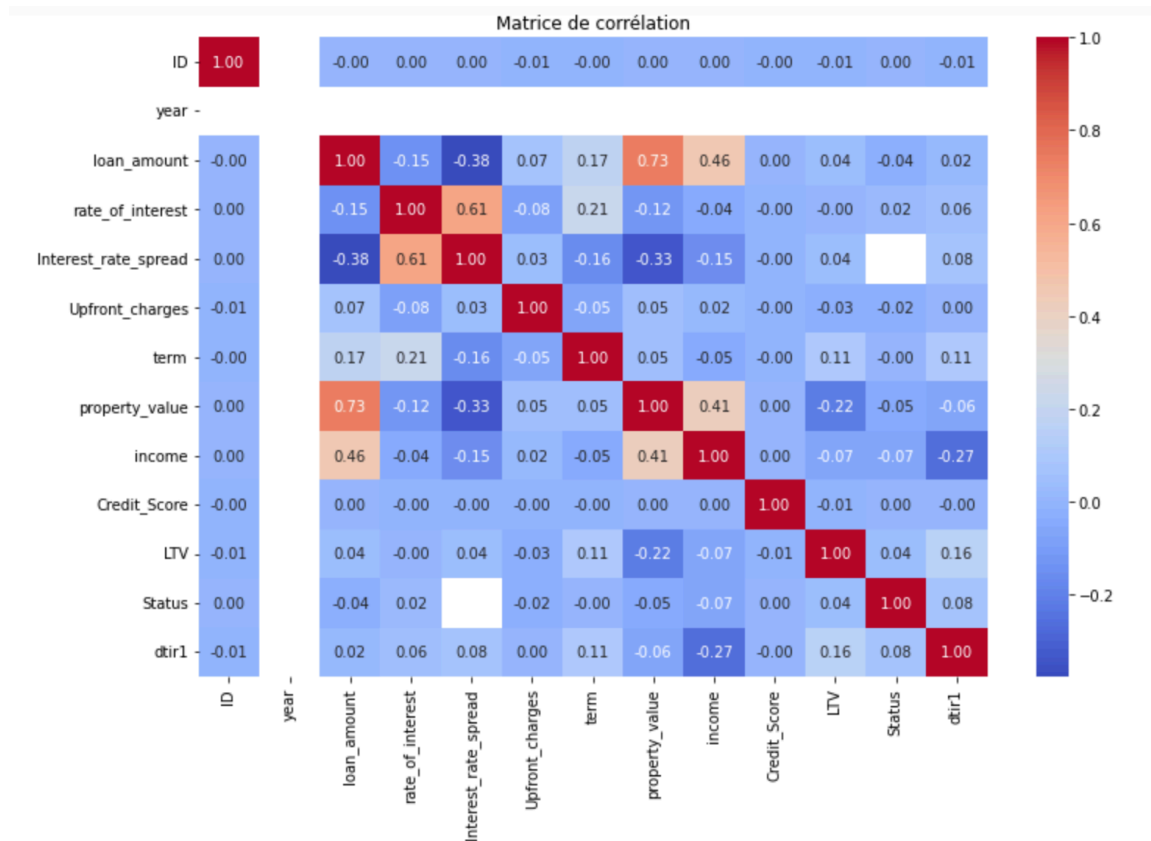
Cependant, certaines variables présentent des statistiques Khi-deux beaucoup plus élevées, indiquant une relation plus forte avec le défaut de paiement, notamment le `credit_type`, `lump_sum_payment` (le mode de paiement forfaitaire), `Neg_ammortization` (l'amortissement négatif), `co-applicant_credit_type` (co-emprunteur), `submission_of_application` (soumission du dossier). Ces variables auront ainsi un grand effet prédictif pour le modèle.

	Variable	Chi2 Statistic	P-value	Relation significative
15	credit_type	52135.280705	0.000000e+00	Oui
10	lump_sum_payment	5237.827442	0.000000e+00	Oui
8	Neg_ammortization	3610.208104	0.000000e+00	Oui
16	co-applicant_credit_type	3092.392571	0.000000e+00	Oui
18	submission_of_application	2171.709755	0.000000e+00	Oui
3	loan_type	1309.958143	3.517253e-285	Oui
7	business_or_commercial	1272.807998	9.172191e-279	Oui
1	Gender	1043.600850	6.260048e-226	Oui
0	loan_limit	427.398487	5.985647e-95	Oui

4) MATRICE DE CORRÉLATION ET ANALYSE VIF

La matrice de corrélation calculée sur l'ensemble des variables numériques montre que les coefficients de corrélation avec la variable cible Status sont globalement faibles. La variable la plus corrélée est dtir1 (ratio d'endettement). Ce résultat indique qu'aucune variable numérique seule n'explique fortement le défaut de paiement de manière linéaire, ce qui renforce l'intérêt des algorithmes non-linéaires comme les random forest et le gradient boosting.

L'analyse du Facteur d'Inflation de la Variance (VIF) confirme l'absence de multicollinéarité problématique dans le dataset. La variable year affiche un VIF élevé car elle ne présente aucune variabilité (valeur unique), ce qui conduit à son exclusion du dataset : une variable constante n'apporte aucune information prédictive.



5) DÉTECTION DES OUTLIERS

La détection des outliers a été réalisée via la méthode IQR (Interquartile Range) : une observation est considérée comme aberrante si elle se trouve en dehors de l'intervalle ($Q1 - 1,5 \times IQR$; $Q3 + 1,5 \times IQR$). Des boxplots et des histogrammes ont été générés pour chaque variable numérique.

L'analyse confirme la présence d'un nombre important d'outliers, particulièrement sur les variables income et property_value. Ces valeurs extrêmes sont cohérentes avec la réalité: les revenus et les valeurs immobilières présentent naturellement une forte dispersion. Ces outliers n'ont pas été supprimés mais ont orienté le choix du scaler vers des approches robustes aux valeurs extrêmes lors du prétraitement.

III. DATA PRÉ-PROCESSING

1) DÉTECTION ET CORRECTION DU DATA LEAKAGE

Lors de la première exécution de la boucle d'entraînement, des performances anormalement élevées ont été observées : la quasi-totalité des modèles affichaient un AUC-ROC de 1,000 y compris des algorithmes simples comme l'arbre de décision et AdaBoost. Ce résultat, physiquement impossible sur un dataset de crédit a révélé la présence d'un data leakage.

Pour confirmer cette hypothèse, un arbre de décision à profondeur 1 a été entraîné afin d'identifier une éventuelle variable dominante unique. L'AUC obtenue était de 0,7495, ce qui est normal et excluait la présence d'une seule variable quasi-parfaite. La cause réelle a été identifiée grâce à l'analyse des features importance du random forest, qui a révélé une concentration anormale du pouvoir prédictif sur trois variables :

- rate_of_interest (taux d'intérêt) ;
- Interest_rate_spread (marge du taux d'intérêt) ;
- Upfront_charges (frais de dossier initiaux).

Ces trois variables constituent un leakage temporel, leur présence dans le dataset permettait aux modèles de lire indirectement la cible, faussant l'ensemble des résultats. Ces trois colonnes ont été supprimées du dataset avant tout split ou entraînement, et le projet a été entièrement réentraîné sans ces variables.

2) FEATURE ENGINEERING

Après suppression des colonnes causant le data leakage, les étapes suivantes ont été appliquées :

- Suppression de l'identifiant ID : variable sans valeur prédictive ;
- Suppression des colonnes constantes (variable year notamment) : une variable sans variabilité n'apporte aucune information supplémentaire au modèle ;
- Feature Engineering : création de la variable income_loan_ratio : ce ratio (revenu annuel / montant du prêt) synthétise la capacité de remboursement relative du client et enrichit l'information existante sans introduire de leakage

3) PIPELINE DE PRÉ-PROCESSING

On a construit un pipeline de pré-processing automatisé avec les classes Pipeline et ColumnTransformer de Scikit-Learn. Cette approche garantit que les mêmes transformations seront appliquées de manière cohérente lors de l'entraînement et de la prédiction sur de nouvelles données.

Pipeline pour les variables numériques

Le pipeline des variables numériques englobe principalement ces 2 étapes :

- SimpleImputer (strategy='median') : imputation des valeurs manquantes par la médiane, insensible aux valeurs extrêmes (outliers) ;
- StandardScaler() : normalisation des variables numériques pour les ramener à une moyenne de 0 et un écart-type de 1. Indispensable pour les algorithmes sensibles aux échelles (comme la régression logistique, KNN et MLP).

Pipeline pour les variables catégorielles

Le pipeline des variables catégorielles englobe également ces 2 étapes :

- SimpleImputer(strategy='most_frequent') : imputation des valeurs manquantes par la modalité la plus fréquente ;
- OrdinalEncoder: encodage ordinal des variables catégorielles. Ce choix est justifié par l'efficacité de ce type d'encodage vu qu'il évite d'ajouter d'autre variable et d'augmenter la dimension du dataset que produirait un encodage One-Hot.

4) SÉPARATION TRAIN-TEST

La séparation des données en ensemble d'entraînement et ensemble de test a été réalisée avec la fonction train_test_split de Scikit-Learn, avec les paramètres suivants :

- test_size=0.20 : 20 % des données sont réservées pour l'évaluation finale, 80 % pour l'entraînement ;
- random_state=42 : garantit la reproductibilité des résultats ;

- stratify=y : garantit que les proportions des 2 classes seront cohérentes dans l'ensemble d'entraînement et l'ensemble de test. Sans cette option, un découpage aléatoire pourrait produire des proportions déséquilibrées et biaiser l'évaluation.

Cette séparation stratifiée est essentielle dans le contexte d'un dataset déséquilibré pour s'assurer que le modèle est évalué dans des conditions représentatives de la distribution réelle des classes du dataset.

IV. CONSTRUCTION ET COMPARAISON DES MODÈLES ML

Onze algorithmes ont été testés dans un pipeline complet (preprocessing + classifieur) avec **validation croisée StratifiedKFold 5-folds**. Le classement final est basé sur le **CV Mean AUC**, plus robuste qu'une simple évaluation sur le test set.

	Modèle	Train Accuracy	Test Accuracy	F1 Score	AUC-ROC	\
7	XGBoost	0.9103	0.9013	0.7648	0.8977	
3	Random Forest	1.0000	0.8949	0.7432	0.8893	
5	Gradient Boosting	0.8884	0.8937	0.7357	0.8849	
8	Bagging	0.9993	0.8940	0.7483	0.8824	
4	Extra Trees	1.0000	0.8795	0.6943	0.8807	
6	AdaBoost	0.8761	0.8805	0.7014	0.8693	
10	MLP Neural Network	0.9114	0.8864	0.7429	0.8794	
2	Decision Tree	0.9336	0.8760	0.7151	0.8172	
9	KNN	0.8700	0.8579	0.6193	0.7997	
0	Logistic Regression	0.7760	0.7803	0.3167	0.7295	
1	Naive Bayes	0.7647	0.7684	0.1563	0.7142	

Trois modèles se distinguent en tête du classement : **XGBoost**, **Gradient Boosting** et **Random Forest**.

Une analyse du **biais-variance** a été conduite en calculant le gap entre la précision sur l'ensemble d'entraînement et sur l'ensemble de test. Un gap supérieur à 0,05 signale un surapprentissage. Random Forest présente un gap légèrement élevé, mais reste candidat au tuning car cet overfitting peut être corrigé en limitant la profondeur des arbres et en augmentant le nombre minimum de samples par feuille.

XGBoost et Gradient Boosting affichent un bon équilibre.

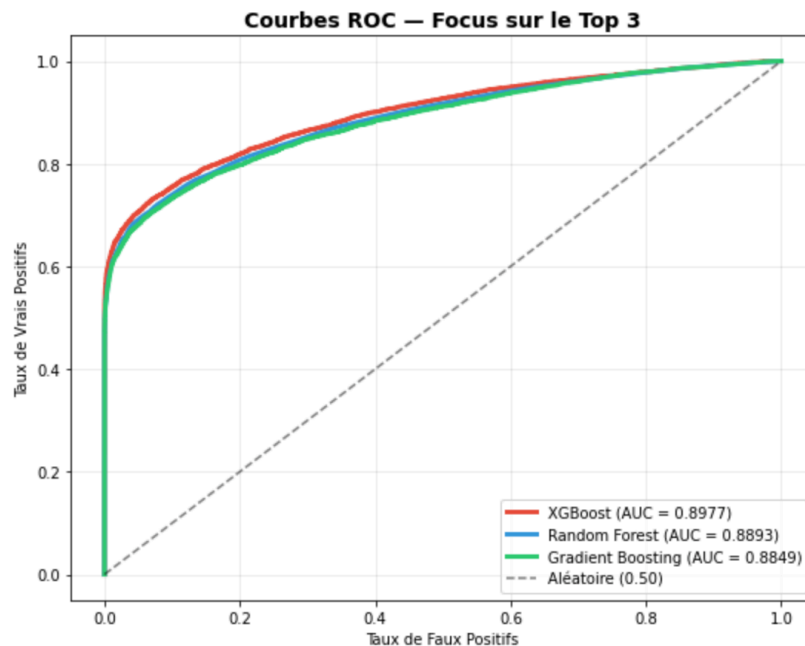
	Modèle	Train Accuracy	Test Accuracy	Gap (Variance)	Diagnostic
5	Gradient Boosting	0.8884	0.8937	-0.0053	Bon Équilibre (Généralise bien)
6	AdaBoost	0.8761	0.8805	-0.0044	Bon Équilibre (Généralise bien)
0	Logistic Regression	0.7760	0.7803	-0.0043	Bon Équilibre (Généralise bien)
1	Naive Bayes	0.7647	0.7684	-0.0037	Bon Équilibre (Généralise bien)
7	XGBoost	0.9103	0.9013	0.0090	Bon Équilibre (Généralise bien)
9	KNN	0.8700	0.8579	0.0121	Bon Équilibre (Généralise bien)
10	MLP Neural Network	0.9114	0.8864	0.0250	Bon Équilibre (Généralise bien)
2	Decision Tree	0.9336	0.8760	0.0576	Variance Élevée (Overfitting)
3	Random Forest	1.0000	0.8949	0.1051	Variance Élevée (Overfitting)
8	Bagging	0.9993	0.8940	0.1053	Variance Élevée (Overfitting)
4	Extra Trees	1.0000	0.8795	0.1205	Variance Élevée (Overfitting)

V. TUNING DES MEILLEURS MODÈLES ET CHOIX FINAL

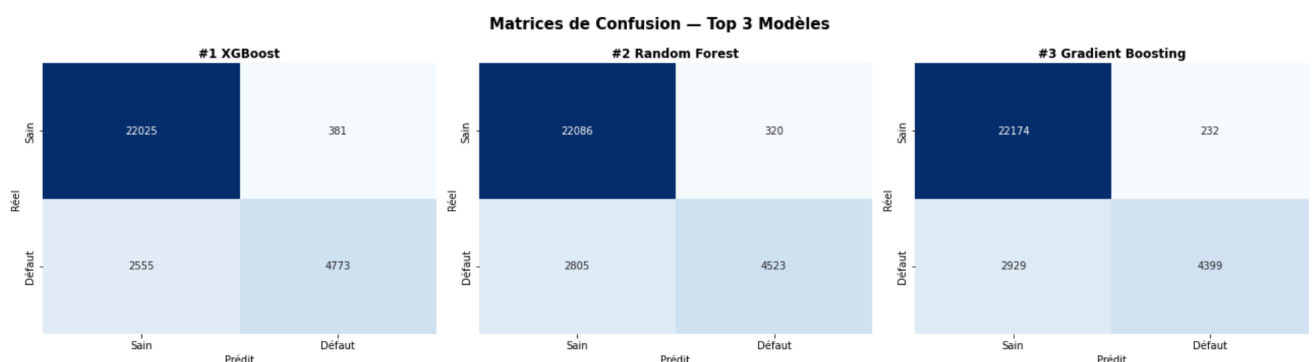
1) SELECTION DES 3 MEILLEURS MODÈLES

Les trois modèles retenus pour le tuning sont **XGBoost**, **Random Forest** et **Gradient Boosting**, sélectionnés sur la base du CV Mean AUC. Le Random Forest, malgré son overfitting initial, est candidat au tuning car on peut régler les paramètres de la limitation de max_depth et min_samples_split.

Les courbes ROC (Receiver Operating Characteristic) des trois meilleurs modèles ont été tracées sur un même graphique. Chaque courbe représente le compromis entre le taux de vrais positifs et le taux de faux positifs pour différents seuils de décision. L'aire sous la courbe (AUC) montre ce compromis : une valeur de 1 correspond à un modèle parfait, et 0,5 à un modèle aléatoire. **La courbe ROC de XGBoost (en rouge) est la plus haute.** Cela signifie qu'à n'importe quel niveau de risque choisi par la banque, XGBoost est le modèle qui détecte le plus de clients en défaut tout en faisant le moins d'erreurs sur les clients sains.



XGBoost présente également le nombre de faux négatifs le plus bas (2 555), ce qui en fait le modèle le plus sûr du point de vue de la gestion du risque de crédit. Gradient Boosting minimise les faux positifs (232), ce qui en fait l'option préférable si l'objectif est de maximiser l'acceptation des bons clients.



2) RÉSULTAT DU TUNING ET CHOIX DU MODÈLE FINAL

L'étape de tuning vise à affiner les hyperparamètres des trois meilleurs modèles afin d'extraire le maximum de performance. On utilise la méthode RandomizedSearchCV qui permet d'échantillonner aléatoirement un nombre fixe de combinaisons (n_iter=20) offrant ainsi le meilleur compromis entre temps

de calcul et gain de performance. La recherche est évaluée via une validation croisée à 3 plis (cv=3) avec le critère roc_auc, et le meilleur pipeline complet (prétraitement + modèle optimisé) est sauvegardé.

```
search = RandomizedSearchCV(pipe, param_grids[name], n_iter=20, scoring='roc_auc', cv=3, n_jobs=2,
                             random_state=42)
```

Les hyperparamètres auxquels on va appliquer le tuning sont :

XG BOOST :

- le nombre d'estimateurs (n_estimators : 100, 300, 500) ;
- le taux d'apprentissage (learning_rate : 0,01, 0,05, 0,1) ;
- la profondeur maximale des arbres (max_depth : 3, 5, 7) ;
- le taux de sous-échantillonnage (subsample : 0,8, 1,0).

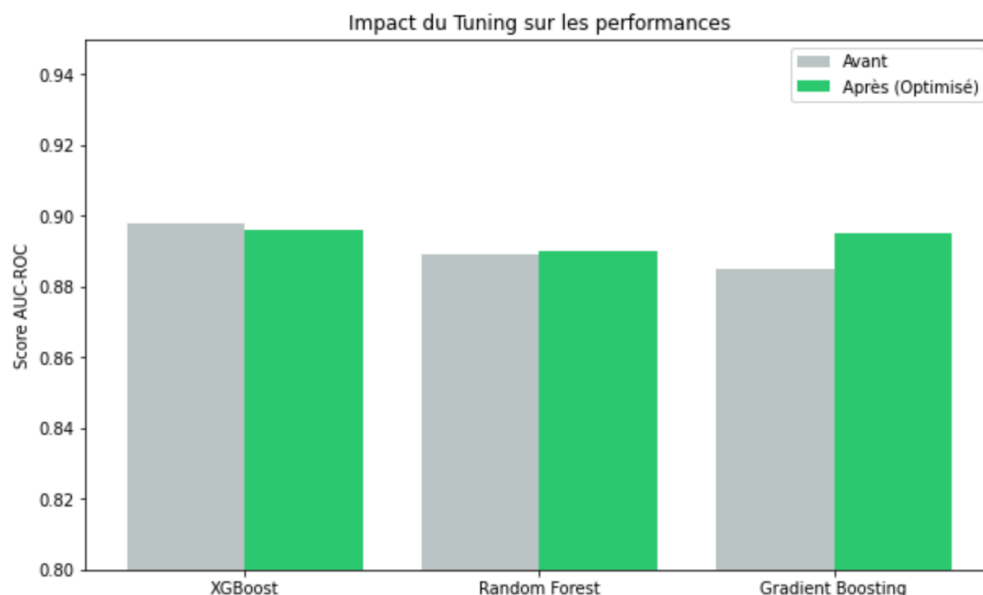
RANDOM FOREST :

- le nombre d'arbres (n_estimators : 100, 200, 300) ;
- la profondeur maximale (max_depth : 10, 20, None) ;
- le nombre minimum d'échantillons pour une division (min_samples_split : 2, 5, 10).

GRADIENT BOOSTING :

- le nombre d'estimateurs (n_estimators : 100, 200) ;
- le taux d'apprentissage (learning_rate : 0,05, 0,1) ;
- la profondeur maximale (max_depth : 3, 5).

Les résultats de ce tuning confirment la pertinence des choix initiaux. Pour XGBoost, une légère baisse est observée après tuning, indiquant que le modèle était déjà bien configuré par défaut et qu'il est difficile d'améliorer son score initial de 0,898. Pour Random Forest, le score est quasi-identique avant et après tuning, ce qui confirme sa robustesse et sa stabilité. Pour Gradient Boosting, une nette amélioration est constatée, passant d'environ 0,885 à 0,895. Malgré ces ajustements, XGBoost conserve la première place avec un AUC-ROC de 0,8977 après optimisation.



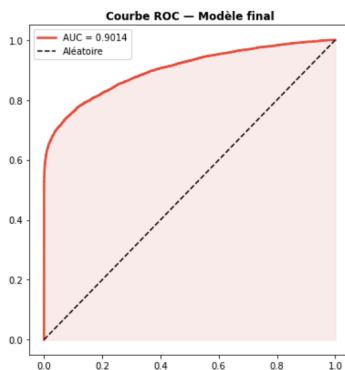
VI. EVALUATION COMPLÈTE DU MODÈLE FINAL

1) RAPPORT DE CLASSIFICATION

À l'issue du tuning, XGBOOST est sélectionné comme modèle final. Les métriques finales calculées montrent un AUC-ROC de 0,90 avec un gap d'overfitting très faible (2.2%) ce qui signifie que le modèle

est 90% plus efficace qu'un choix aléatoire. Il est particulièrement fiable lorsqu'il détecte un risque (94% de précision), ce qui sécurise les décisions de refus de prêt.

La courbe ROC monte très rapidement vers le coin supérieur gauche, prouvant que le modèle identifie les profils les plus risqués avec une grande précision et très peu de faux positifs initiaux.



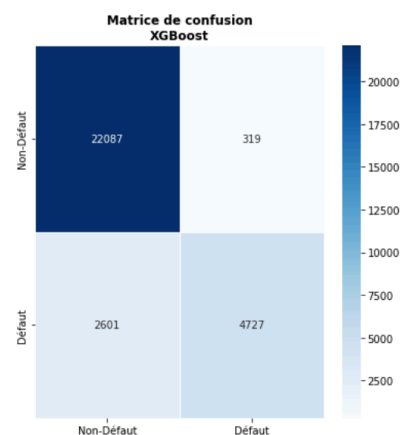
Rapport de classification :

	precision	recall	f1-score	support
Sain (0)	0.89	0.99	0.94	22406
Défaut (1)	0.94	0.65	0.76	7328
accuracy			0.90	29734
macro avg	0.92	0.82	0.85	29734
weighted avg	0.91	0.90	0.90	29734

2) ANALYSE DE LA MATRICE DE CONFUSION

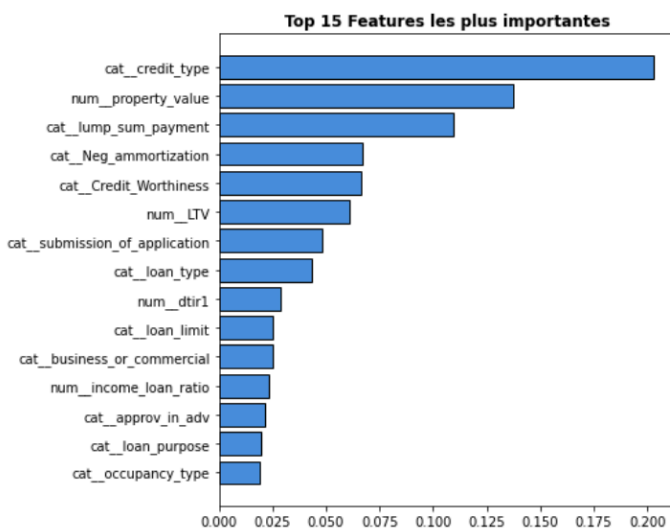
La matrice de confusion finale sur l'ensemble de test présente les résultats suivants :

- Vrais Négatifs (TN) : 22 087, clients sains correctement identifiés,
- Vrais Positifs (TP) : 4 727, clients en défaut détectés, représentant les pertes financières évitées ;
- Faux Positifs (FP) : 319, clients sains refusés à tort (erreur commerciale) ;
- Faux Négatifs (FN) : 2 601, clients défaillants non détectés (erreur financière)



3) FEATURE IMPORTANCE

L'analyse des feature importance du modèle XGBoost final identifie les trois variables les plus déterminantes dans la prédiction du risque de défaut :



- credit type (variable la plus importante) : le type de crédit est le premier indicateur de risque. Certains type de crédit ont plus de probabilité de défaut de paiement que d'autres;

- property value : la valeur du bien immobilier représente la garantie de la banque. Une valeur faible ou instable augmente le risque de défaut de remboursement ;

- lump sum payment : la capacité ou l'incapacité à effectuer des remboursements forfaitaires traduit la solidité financière globale du client.

Ces trois facteurs sont parfaitement cohérents avec la logique métier du risque de crédit immobilier, ce qui

valide la pertinence du modèle d'un point de vue financier.

VII. DÉPLOIEMENT – STREAMLIT

1) SÉRIALISATION DU PIPELINE

Le pipeline complet final (prétraitement + modèle XGBoost) est sauvegardé sous forme de fichier binaire à l'aide de la librairie joblib. Cette approche garantit que l'intégralité du pipeline est conservé : lors d'une prédiction sur un nouveau client, les données brutes passeront automatiquement par les mêmes étapes d'imputation, d'encodage et de normalisation qu'à l'entraînement.

2) GUIDE D'UTILISATION DE L'APPLICATION

L'application streamlit transforme le modèle XGBoost en interface interactive. Elle est développée en Python/Streamlit et charge le pipeline complet. L'interface est personnalisée via l'injection de CSS et de cartes KPI.

L'utilisateur saisit le profil client dans la barre latérale (montant du prêt, valeur du bien, revenu, score de crédit, ratio d'endettement, variables catégorielles) et obtient instantanément un score de risque sur une jauge à trois zones : **verte** (0-30 %), **orange** (30-50 %), **rouge** (50-100 %). Des facteurs d'alerte explicatifs s'affichent automatiquement si certains seuils critiques sont dépassés :

if credit_score < 650: #mauvais payeur potentiel

insights.append("Le faible score de crédit (Credit Score) dégrade la note globale.")

if LTV > 80: #le ratio entre le montant du prêt et la valeur de la maison

insights.append("Le ratio d'emprunt sur la valeur du bien (LTV) est excessivement haut.")

if dtir1 > 45: #C'est le taux d'endettement (les mensualités par rapport aux revenus)

insights.append("Le taux d'endettement de l'acheteur (Debt-To-Income) dépasse le seuil d'alerte.")

- Lancer l'application avec streamlit ;
- Dans la sidebar, renseigner les données financières du client : montant du prêt, valeur du bien (le LTV se calcule automatiquement), revenu annuel, score de crédit et ratio d'endettement ;
- Compléter les caractéristiques du dossier : type de crédit, type de bien, région, présence d'un co-emprunteur, etc.
- Cliquer sur le bouton d'évaluation, la jauge de risque s'affiche avec la probabilité de défaut et les facteurs d'alerte détectés ;
- Possibilité de prédire sur un fichier csv.

RÉALISÉ PAR : FARES NOUHAILA.